

scanner_agent v0.2.0

Handoff Notes — May 2, 2026 | Phase 2 DB Scanner Active

This document captures the full state after two work sessions on May 2, 2026. Phase 1 (local file scanner) is complete and stable. Phase 2 (database audit scanner) is now active: **db_scanner.py**, **record_matcher.py**, and 11 new tests are implemented and verified. The project vision is confirmed: a civic audit intelligence agent for Puerto Rico government databases (CEE, CESCO, Registro Demográfico) aimed at eliminating duplicate voter registrations, duplicate IDs, deceased persons in active rolls, and creating a unified clean identity database.

1 Project Vision (Confirmed)

The scanner_agent is a **Puerto Rico civic audit intelligence agent**. Its purpose is to detect and help eliminate duplicate records across three government databases:

- **CEE** — Comisión Estatal de Elecciones (voter registration)
- **CESCO** — Driver license and Real ID database
- **Registro Demográfico** — Births, deaths, marriages (ground truth)

The matching pipeline compares records by nombre, apellidos, fecha de nacimiento, dirección, and seguro social. It produces four types of findings: **EXACT_DUPLICATE** (same canonical hash), **NEAR_DUPLICATE** (fuzzy name match + DOB/SSN cross-check), **POSSIBLE_DECEASED** (edad > 100), and **ANOMALY** (impossible DOB). All findings are human-reviewed — no automatic deletion. The target is 95%+ depuration leading to a single clean unified identity database and a unique Puerto Rico ID per citizen.

2 What Was Built Today

Session 1 — Cleanup & Phase 1 Hardening

- Fixed README bug: --root-path → --path
- normalize_target_paths: warn+skip instead of raise+abort on missing paths
- Terminal summary block printed after every local scan
- JSON reports enriched with meta block: timestamp, targets, duration
- 4 new test files → 9 logic checks (local_scanner edge cases + report structure)
- TESTING.md rewritten with coverage table; Project_Structure.txt updated

Session 2 — Phase 2 DB Scanner

- **models.py extended**: RowRecord + RecordMatchGroup added alongside existing FileRecord/DuplicateGroup
- **db_scanner.py (ACTIVE)**: SQLite connector with batched row fetching, field_map config, SHA-256 row hashing, accent-stripping normalization

- **record_matcher.py (ACTIVE)**: 3-pass matching engine — exact hash, fuzzy name+DOB (difflib), SSN cross-match, deceased/anomaly flagging
- **mock_cee.db**: 470 synthetic Puerto Rico records (350 clean + 50 exact dup pairs + 30 near-dup + 20 DOB-off + 15 possible deceased + 5 anomalies)
- **test_db_scanner.py**: 11 tests covering scan fields, hash, normalization, exact dups, near-dups, deceased, anomaly, full pipeline
- **CLI v0.2.0**: `--scanner db + --db + --table + --source-alias`
- **Rich terminal output**: colored tables, confidence, flag colors — degrades gracefully if rich not installed
- **pyproject.toml v0.2.0**: rich added as core dep; google-genai moved to optional [ai] extras for Phase 3

3 Current Status

Componente	Estado	Notas
Phase 1 — Local file scanner	COMPLETO	SHA-256, exclusions, JSON reports
Phase 1 — CLI (local)	COMPLETO	--path, --output-dir, terminal summary
Phase 1 — Test suite (7 checks)	COMPLETO	All passing
Phase 2 — RowRecord / RecordMatchGroup	NUEVO	models.py extended
Phase 2 — db_scanner.py (SQLite)	ACTIVO	Batched, field_map, hashing, normalization
Phase 2 — record_matcher.py	ACTIVO	Exact + fuzzy + SSN + deceased + anomaly
Phase 2 — mock_cee.db (470 rows)	ACTIVO	470 records, 4 anomaly types
Phase 2 — test_db_scanner.py (11)	ACTIVO	All 11 checks passing
Phase 2 — CLI --scanner db	ACTIVO	Full DB audit flow wired
Phase 2 — Rich terminal	ACTIVO	Colored tables, degrades gracefully
Phase 2 — Web review dashboard	PENDIENTE	Flask + HTML — next priority
Phase 2 — CESCO field_map	PENDIENTE	Add CESCO-specific column mapping
Phase 3 — Gemini AI integration	PENDIENTE	After DB scanner fully validated
Phase 3 — Registro Demográfico	PENDIENTE	Cross-DB matching pipeline

4 Mock CEE Database Results

Running the full scan against mock_cee.db (470 synthetic PR records):

Metric	Value	Notes
--------	-------	-------

Registros escaneados	470	350 clean + 120 seeded anomalies
Duplicados exactos	47 grupos	94 registros involucrados
Near-duplicates	42 pares	Fuzzy name typos + DOB-off-by-1yr
Posibles fallecidos	10 registros	edad > 100
Anomalías de edad	10 registros	DOB imposible o en el futuro

5 Next Steps

Step 1 — Activate on Windows (immediate)

In PowerShell with .venv active:

```

pip install rich
pip install -e .
pytest -v
python tests/create_mock_db.py # regenerates tests/data/mock_cee.db
python -m scanner_agent --scanner db --db "sqlite:///tests/data/mock_cee.db"
--source-alias CEE

```

Expected: Rich colored table showing 47 exact dup groups, 42 near-dups, 10 deceased, 10 anomalies.

Step 2 — Add CESCO field_map

Once CESCO table structure is known, add its column mapping to db_scanner.py or as a named preset in a new field_maps.py module. Then run the same pipeline against CESCO and compare with CEE results.

Step 3 — Web Review Dashboard (Flask)

This is the critical interface for human review. Minimal viable version:

- One HTML page per finding flag (EXACT, NEAR, DECEASED, ANOMALY)
- Side-by-side record comparison for duplicate pairs
- Buttons: Confirm Duplicate / Not a Duplicate / Needs More Info
- Decisions saved to review_log.json with timestamp + reviewer
- Progress bar: X of N findings reviewed

Flask + Jinja2 — pure Python, no frontend framework needed for MVP.

Step 4 — Cross-DB matching (CEE × CESCO)

After CEE and CESCO are individually deputed to 95%+, run the combined scan: compare normalized names + DOB across both tables. The goal is to find people registered in both systems with conflicting data, and progressively converge to a single identity record.

6 Quick-Start Commands

1. Activate env

```
.venv\Scripts\Activate.ps1
```

2. Run all tests

```
pytest -v
```

3. Lint check

```
ruff check .
```

4. Local file scan

```
python -m scanner_agent --path ".\sandboxtest_test"
```

5. DB audit scan

```
python -m scanner_agent --scanner db --db "sqlite:///tests/data/mock_cee.db"  
--source-alias CEE
```

6. Rebuild mock DB

```
python tests/create_mock_db.py
```

7 Context Restore Phrase

We left off after May 2 — two sessions. Phase 1 (local scanner) is complete. Phase 2 is active: db_scanner.py + record_matcher.py are working, 12 total checks pass, mock_cee.db has 470 synthetic PR records producing 47 exact dups + 42 near-dups + 10 deceased + 10 anomalies. CLI v0.2.0 supports --scanner db. Rich terminal output is wired. Project vision: PR civic audit agent — CEE + CESCO + Registro Demográfico → unique citizen ID. Next: pip install rich, run pytest, run the DB scan against mock_cee.db, then build the Flask web review dashboard.